Joc de Rol. Curs 2025-26

Joan Gerard Camarena Estruch

Continguts

1 Joc de Rol **3**

2 Generalitats **4**
2.1 Entregues per Fases usant Git i GitHub 4
2.1.1 Inici del repositori 4
2.1.2 Treball diari (Commits normals) 4
2.1.3 Final de fase i etiquetar 5
2.1.4 Recordatori: Que fer quan he acabat una fase? 5
2.1.5 I que farà el profeessor per a avaluar-me? 5
2.1.6 Resum 6

3 Fase 1. Herència **7**
3.1 Explicació general de la pràctica 7
3.2 Tasques a efectuar: 8
3.3 Fi de la Fase 1 9

4 Fase 2. Donant cos als jugadors **10**
4.1 Explicació 10
4.2 Es demana 10
4.3 Fi de la Fase 2 12

5 Fase 3. Sobrecàrrega i Polimorfisme **13**
5.1 Explicació 13
5.2 Es demana 14
5.3 Fi de la Fase 3 14

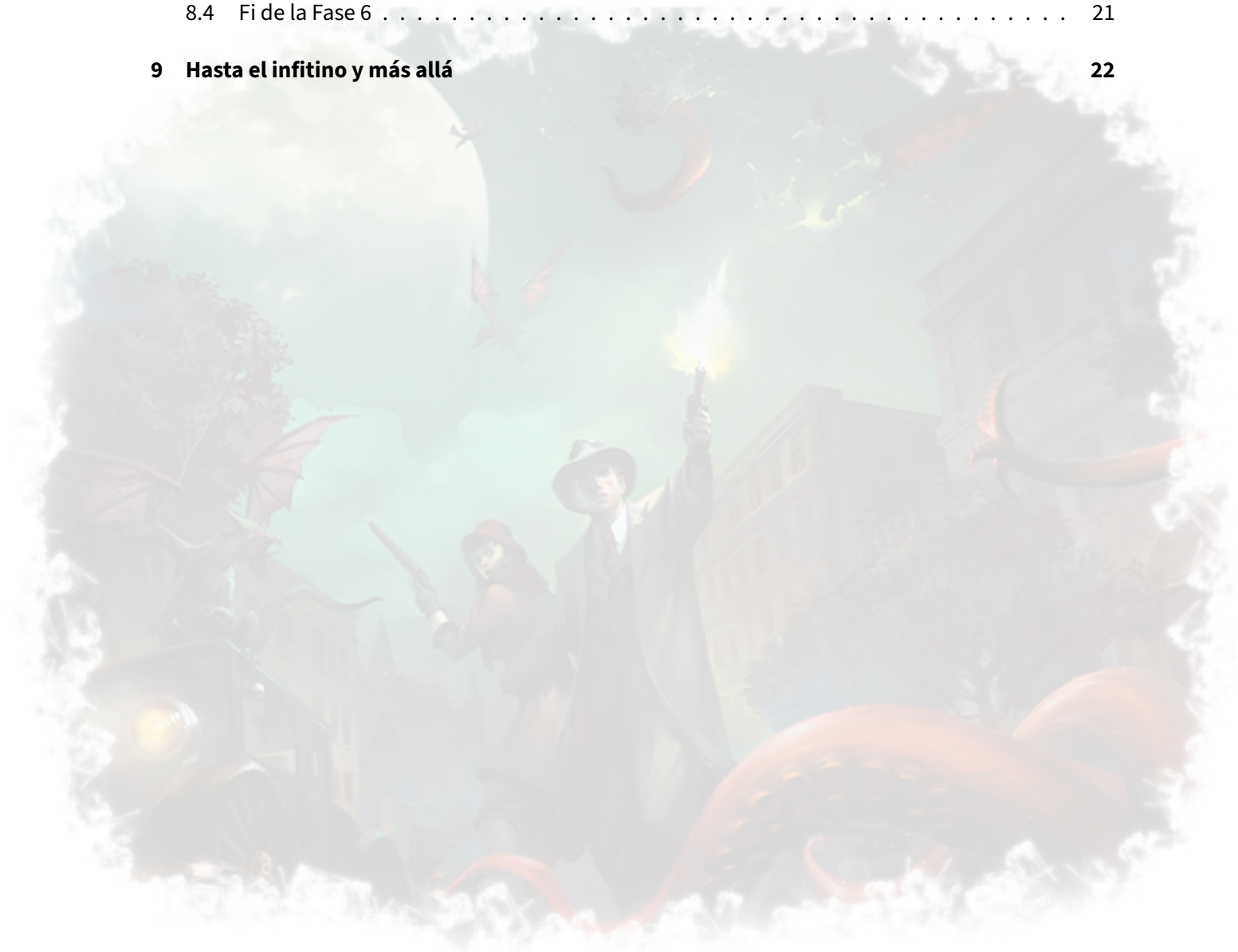
6 Fase 4. Relacions entre classes **15**
6.1 Explicació 15
6.2 Es demana 15
6.3 Fi de la Fase 4 16

7 Fase 5. Més relacions **17**
7.1 Explicació 17
7.2 Es demana 17
7.3 Fi de la Fase 5 18

8 Fase 6. Configuració del programa principal i Excepcions **19**
8.1 Explicació 19

8.2 Es demana 19
8.2.1 Menú Jugadors 19
8.2.2 Menú Equips 20
8.2.3 Menú Ítems 20
8.2.4 Jugar 21
8.3 Excepcions 21
8.4 Fi de la Fase 6 21

9 Hasta el infitino y más allá 22



1 Joc de Rol

Figura 1: Joc de Rol

© Arkham Horror - El joc de Rol. Fantasy Flight Games

2 Generalitats

En aquesta pràctica anem a fer-la durant 3 entregues, de manera que anirem afegint-li funcionalitat a la mateixa. Per aixòp farem un poquet i acabem, farem altre poquet més i parem, i així fins a completar les 7 fases que consta la totalitat de la pràctica.

Per a desenvolupar-ho i no perdre res ho podem fer de dos maneres:

1. 7 projectes distints en 7 carpetes distintes. Quan acabem la primera copiem i apeguem i a partir d'aquesta còpia fem la segona. D'aquesta manera quan acabem la fase *n* és l'inici de la fase *n+1*
2. Fer servir **GIT**. De manera que ens posem a programar. Quan acabem cada fase fem un *commit* donant-li com a nom la fase en la que estem. Això ens permetrà tornar a cadascuna de les fases anteriors.

2.1 Entregues per Fases usant Git i GitHub

Aquest projecte es desenvoluparà i s'avaluarà en 3 fases distintes. Per que poder corregir les fases de manera independent, a banda de si el projecte final ja està acabat, anem a utilitzar les Etiquetes (`Tags`) de Git. Una etiqueta funciona com un **marcapàgines** exacte en la història del codi.

2.1.1 Inici del repositori

Pas 1: Crea el teu repositori a github.

Pas 2: Convidar al professor al repositori. Per a poder veure el vostre codi i avaluar-vos, heu de donar accés al vostre repositori en GitHub:

1. Entreu en el vostre repositori en GitHub.
2. Feu clic a la pestanya `Settings` (Ajustos).
3. En el menú de l'esquerra, feu clic en `Collaborators` (Col·laboradors).
4. Feu clic en el botó `Add people` (Afegir persones).
5. Buscau l'usuari del teu professor de GitHub o el seu correu i envieu la invitació.

2.1.2 Treball diari (Commits normals)

Mentre treballeu en una fase, feu els vostres commits de forma habitual. És recomanable fer commits menuts i freqüents. També donar-li descripcions del canvis que feu en cada commit, per a que després sigui més fàcil recordar què heu fet en cada fase. Per exemple:

```
1 git add .
2 git commit -m "Afegida l'estructura del HTML"
3 git push
```

> **Nota**: Podeu fer tots els commits que vulgueu dins d'una mateixa fase. No hi ha límit.

Ara ja estàs en disposició de continuar.

2.1.3 Final de fase i etiquetar

Quan considereu que una fase està 100% acabada i a punt per que l'avalúe, heu de posar una **etiqueta** a l'últim commit d'eixa fase. Imaginau que haueu acabat la Fase 1. Heu de:

1. Crear l'etiqueta
2. Pujar l'etiqueta a GitHub (¡**MÉS IMPORTANT**!). El comandament `git push` normal **NO puja les etiquetes** a GitHub. Si només feu un push normal, l'etiqueta es quedarà en l'ordinador vostre i el professor no la podrà veure.

```
1 git tag fase-1
2 # Per a les següents entregues, simplement canviareu el número: fase-2,
      fase-3, etc.
3
4 git push origin fase-1
5 # Per a les següents entregues, simplement canviareu el número: fase-2,
      fase-3, etc.
6 # Si preferiu pujar totes les etiquetes que haueu creat de colp, podeu
     usar: git push --tags).
```

2.1.4 Recordatori: Que fer quan he acabat una fase?

El dia que acabeu una fase, els comandos exactes que heu d'executar en orde són:

```
1 git add .
2 git commit -m "Últims retocs de la fase X"
3 git push                  #Puja el codi
4 git tag fase-X            # Crea l'etiqueta d'entrega
5 git push origin fase-X    # Puja l'etiqueta al repo
```

2.1.5 I que farà el profeessor per a avaluar-me?

Com a complement al funcionament dels repositoris pot resultar interessant saber com el professor va a avaluar-vos. El professor, per a avaluar-vos, farà el següent:

Pas 1: Descarregar el repositori (sols la primera vegada) si encara no ho ha fet

```
1 git clone URL_DEL_REPOSITORI_DE_L_ALUMNE  # clona en repo
2 cd nom_del_repositori                     # entra en el repo local
```

Pas 2: Actualitzar els canvis i etiquetes. Viatjar en el temps a la fase a corregir

Si ja tenies el projecte descarregat, assegura't de baixar-te els últims canvis i les noves etiquetes que hagi pujat l'alumne:

```
1 git checkout main          # ens situem al present (última versió)
2 git pull                   # baixem els últims canvis
3 git fetch --tags           # baixem les últimes etiquetes
4 git tag -l                 # llistem les etiquetes disponibles
5 git checkout fase-1        # ens situem a la fase 1 per a corregir-la
```

Al fer això (anar al passat), Git dirà un missatge una mica desagradable dient que estàs en un estat *detached HEAD* (cap separada). Això és normal. Només significa que estàs en mode *només lectura*. Pots executar el codi, provar-lo i veure'l, però no hauries de fer commits nous allà ni modificar-lo.

Pas 4: Tornar al present

Quan acabis de corregir la fase-1 i vulguis tornar a l'estat final del projecte (o preparar-te per a corregir una altra fase), només has de tornar a la branca principal:

```
1 git checkout main
```

2.1.6 Resum

El flux de treball del professor serà el següent:

1. `git fetch --tags`
2. `git checkout fase-1` (corregir fase 1)
3. `git checkout fase-2` (corregir fase 2)
4. ... les fase que calguen
5. `git checkout main` (feina feta).

3 Fase 1. Herència

3.1 Explicació general de la pràctica

Volem programar un joc de rol on es van a crear diversos jugadors. A aquests jugadors els anomenarem de manera genèrica `Player`. Al moment de jugar, es pot triar entre 3 tipus diferents de jugadors: humà (`Human`), guerrer (`Warrior`) o alienígena (`Alien`). Cada personatge té unes característiques pròpies, que són: els punts d'atac, els punts de defensa i els punts de vida i el comportament de cada jugador dependrà de la seua raça.

El joc consistirà en que els jugadors s'ataquen entre ells. Cada vegada que el jugador A ataca el jugador B, passen 2 coses, A colpeja Bi si B encara està viu, B colpeja A. És a dir, cada atac és un intercanvi de colps entre els dos jugadors.

Quan el jugador A colpeja a B es fa la resta entre els punts d'atac del jugador A i els punts de defensa del jugador B. Si el resultat és positiu, el jugador B perd esta quantitat de punts de vida. Si al jugador B encara li queden punts de vida (no està mort) es podrà defendre i colpejar a A. Quan els punts de vida d'un jugador arriben a zero, diem que és mort.

La relació dels diferents tipus de jugadors es mostra en el següent diagrama:

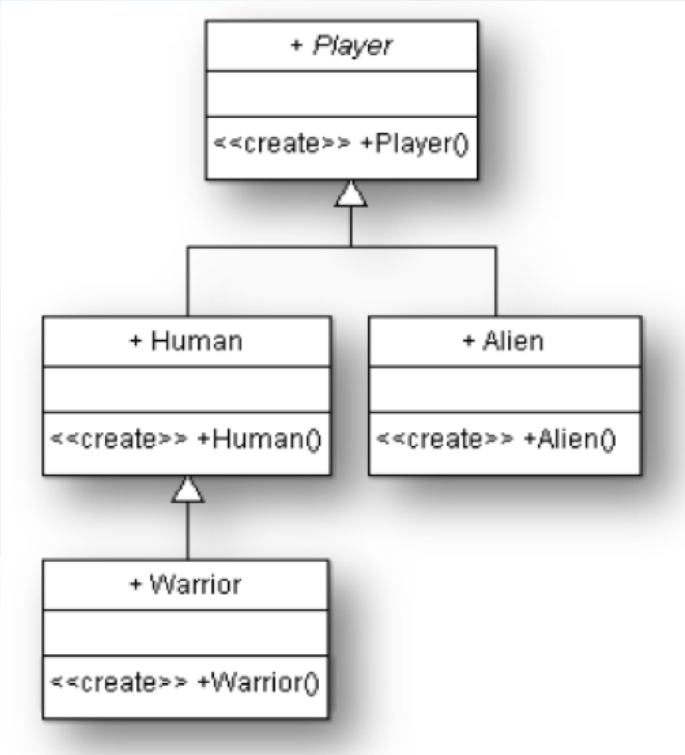


Figura 2: Jugador

Com podem vore, un jugador `Warrior` és una especialització d'un jugador `Human`. `Alien` i `Human` són també una especialització de `Player`. Això ho podem aconseguir mitjançant l'*herència*.

La classe `Player` serà abstracta. És a dir, no podrem crear jugadors (objectes) d'eixa classe, sinó de les especialitzades, les quals es diferencien en:

- `Human` → No tenen bonificacions en defensa ni en atac.
- `Alien` → Tenen bonificacions en cada atac però també penalitzacions en defensa.
- `Warrior` → Poden aguantar més ferides que la resta de jugadors.

3.2 Tasques a efectuar:

1. Crea l'aplicació `JocDeRol` amb els següents paquets (cas de Java).

- `util`: Per a les classes d'utilitats i de lectura escriptura de teclat i pantalla. Pots fer servir les subministrades al llarg del curs pel professor.
- `joc`: Per a les classes `Player`, `Human`, `Alien` i `Warrior`.
- `inici`: Per a la classe `JocDeRol`, amb el programa principal.

2. Implementa en el paquet joc les 4 classes (`Player`, `Human`, `Warrior`, `Alien`), amb les herències que pertoquen i amb els mètodes constructors per defecte corresponents, sense paràmetres. Els constructors, de moment, han de mostrar per pantalla el següent text:

```
CONSTRUCTOR -> He creat un <nom_de_la_classe_corresponent>
```

3. En la classe `JocDeRol` del paquet inici crea la funció `provaFaseV1()`, que ens ajudarà a entendre com funciona el mecanisme de l'herència. Eixa funció ha de crear un objecte de cada tipus (`Human`, `Warrior` i `Alien`). Abans de crear cada objecte avisarà per pantalla el tipus d'objecte que va a crear. En executar-ho, comprovarem que quan es crida al constructor d'una classe, es crida automàticament (i per ordre) a tots els constructors de les classes de les quals hereta.

Notes:

- A l'igual que `provaFaseV1()`, també haureu de crear les funcions `provaFaseV2()`, `provaFaseV3()`, etc. per a cada fase, per a comprovar el funcionament de cada fase. Així, quan acabeu la fase 2, la funció `provaFaseV2()` serà la que us servirà per a comprovar el funcionament de la fase 2, i així successivament.
- `main()` sols cridarà a cada una de les funcions `provaFaseV1()`, `provaFaseV2()`, etc.

> per a comprovar el funcionament de cada fase. Així, quan acabeu la fase 2, main() sols cridarà a `provaFaseV2()`, i així successivament.

3.3 Fi de la Fase 1

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 1. Recorda que el nom de l'etiqueta ha de ser `fase-1`.

4 Fase 2. Donant cos als jugadors

4.1 Explicació

Afegirem els atributs per a guardar els punts d'atac, defensa i vida, així com els mètodes necessaris per a atacar un altre jugador. El diagrama de classes quedarà de la manera següent:

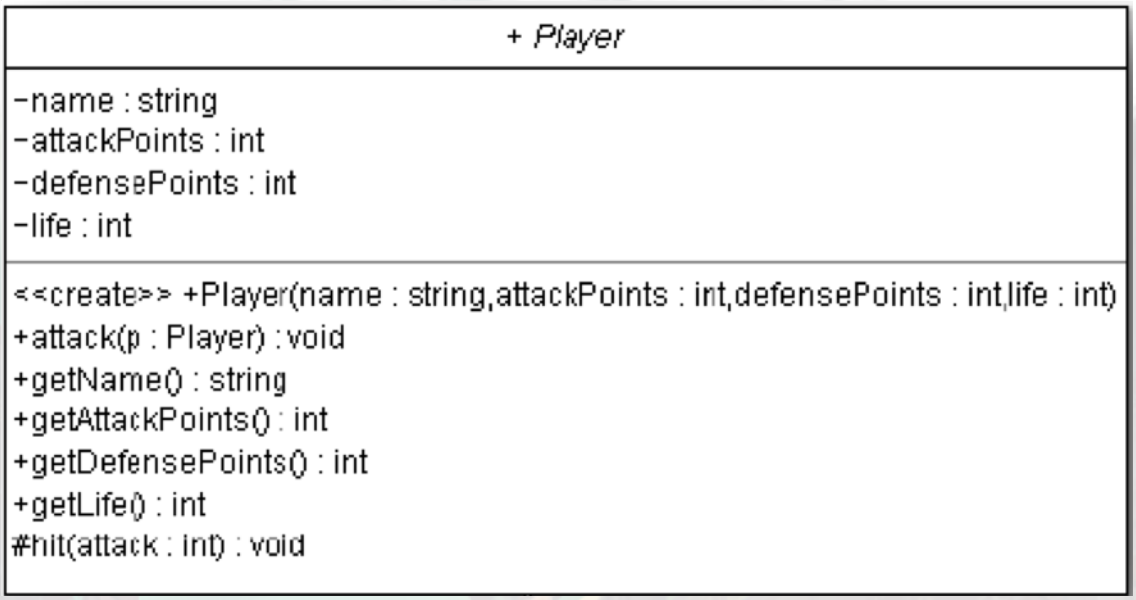


Figura 3: Jugador

Nota: Aquesta representació és del diagrama de classes de UML. A la qual, els camps i/o mètodes s'han de posar com:

- Es posa un − quan l'atribut o mètode és private
- Es posa un + quan l'atribut o mètode és public
- Es posa un # quan l'atribut o mètode és protected

4.2 Es demana

1. Modifica la classe `Player`:

- Afig 4 atributs: `name`, `attackPoints`, `defensePoints`, `life`
- Modifica el constructor per a passar-li els 4 atributs. Caldrà modificar també els constructors de les altres classes hereves.
- Afix els 4 mètodes *getters* corresponents per a consultar cada atribut.

- De moment no calerà afegir els *setters*, ja que no volem que es modifiquen les característiques d'un jugador un cop creat.

2. Sobreescriu el mètode `toString()` de la classe `Object` en la classe `Player`. Retornarà totes les dades del jugador amb el següent format:

```
John Smith PA:13 / PD:8 / PV:39
```

- Afig el mètode void `attack(Player p)`, que s'utilitza quan un jugador vol atacar un altre. Quan un jugador A vol atacar un altre jugador B, es fa la crida `A.attack(B)`. Este mètode consisteix en que B siga colpejat per A (una crida al mètode `hit()` de B) i que, en cas de que B encara estigui amb vida li podrà tornar el colp, i A siga colpejat per B (una crida al mètode `hit()` de A). > Has d'interpretar `hit()` com a rebre un colp. Per això, quan A ataca a B, B és el que rep el colp, i per tant és el que crida al mètode `hit()`. De la mateixa manera, quan B ataca a A, A és el que rep el colp, i per tant és el que crida al mètode `hit()`.
- Afig el mètode void `hit(int attackPoints)`:
  - Serà **`protected`** (només accessible des de la classe i classes hereves), ja que només ha de cridar-se des del mètode `attack`.
  - És cridat des del mètode `attack` (dos vegades) per a dir que un jugador és colpejat per un altre amb tants punts d'atac.
  - El mètode ha de restar tants punts de vida com la diferència entre els punts amb què l'ataquen i els punts de defensa que té. Els punts de vida mai podran ser augmentats ni ser menors que 0.
  - Mostrarà un missatge dient qui és atacat, amb quants punts l'ataquen, amb quants punts es defén, quanta vides tenia, quantes li'n lleven i quantes en tindrà finalment.
- Abans i després de fer estes 2 accions, caldrà mostrar les dades de cadascun dels 2 jugadors. Per exemple:

```
//    ABANS DE L'ATAC:
Atacant: John Smith PA:13 / PD:8 / PV:39
Atacat:  Martian PK PA:27 / PD:2 / PV:32
//   ATAC:
Martian PK és colpejat amb 13 punts i es defén amb 2. Vides: 32 - 11 =
    21
John Smith és colpejat amb 27 punts i es defén amb 8. Vides: 39 - 19 =
    20
//    DESPRÉS DE L'ATAC:
Atacant: John Smith PA:13 / PD:8 / PV:20
Atacat: Martian PK PA:27 / PD:2 / PV:21
```

> Nota: les línies de l'atac les mostra el mètode `hit` en les dos crides corresponents que es fan a aquest mètode.

4. En la classe JocDeRol del paquet inici modifica la funció `provaFase()`, que ens mostre el funcionament de la funció dels atacs.

- Eixa funció ha de crear un objecte de cada tipus (Human, Warrior i Alien).
- Fer alguns atacs entre ells
- Comprovar en l'eixida que les puntuacions estan ben efectuades

4.3 Fi de la Fase 2

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 2. Recorda que el nom de l'etiqueta ha de ser `fase-2`.

5 Fase 3. Sobrecàrrega i Polimorfisme

© Arkham Horror, 3r Edició. Fantasy Flight Games

5.1 Explicació

Com hem dit abans, les classes hereves de `Player` són especialitzacions seues. És a dir: `Player` implementa un comportament genèric que cadascuna de les classes hereves pot modificar. Este canvi de comportament es pot realitzar bé afegint atributs i mètodes propis a la classe hereva o bé tornant a codificar algun dels mètodes de la classe heretada.

El *polimorfime* consistix en utilitzar el mecanisme de redefinició (overriding en anglés). Consistix en redefinir. És a dir: tornar a codificar el comportament heretat d'acord a les necessitats d'especialitació de la classe hereva.

En el nostre cas, volem tornar a codificar els mètodes necessaris a cada classse hereva per a aconseguir el següent comportament:

- Els jugadors de tipus `Human` no podran tindre més de 100 punts de vida i, per tant, s'ha de limitar esta característica al moment de la seua creació.
- Els jugadors de tipus `Alien` embogixen quan ataquen, però obliden la seua defensa. Quan un `Alien` ataca, si no està greument ferit (punts de vida superiors a 20) augmenten en 3 els seus punts d'atac i disminueixen en 3 els seus punts de defensa. Si estan greument ferits (punts de vida iguals o inferiors a 20) es comporten de manera normal.

> Nota: Els punts d'atac queden augmentats també després de l'atac (i també els de la defensa després de defendre), és a dir aquestes modificacions es mantenen.

- Els jugadors de tipus `Warrior`, degut al seu entrenament, tenen una gran agilitat. Si el colp no és superior a 5 punts, aquest queda reduït a 0. El colp (*hit*) és la diferència entre l'atac que sofrix i la defensa del jugador.

5.2 Es demana

1. Codifiqueu les classes `Human`, `Alien` i `Warrior`, sobreescrivint els mètodes necessaris per tal que tinguen el comportament descrit anteriorment.
2. En la classe JocDeRol del paquet inici adapta la funció `provaFase()` per a comprovar el funcionament. Eixa funció ha de crear, almenys, 3 jugadors de diferent tipus i mostrarà les seues dades. També ha d'incloure diversos atacs d'uns a altres.

5.3 Fi de la Fase 3

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 3. Recorda que el nom de l'etiqueta ha de ser `fase-3`.

6 Fase 4. Relacions entre classes

6.1 Explicació

Volem crear equips de jugadors amb la següent característica. **Qualsevol jugador pot afegir-se a un equip**. Un equip, per tant, és un conjunt de jugadors, i cada jugador pot pertànyer a més d'un equip.

El diagrama UML que representa esta associació entre les classes Player i Team és el següent:

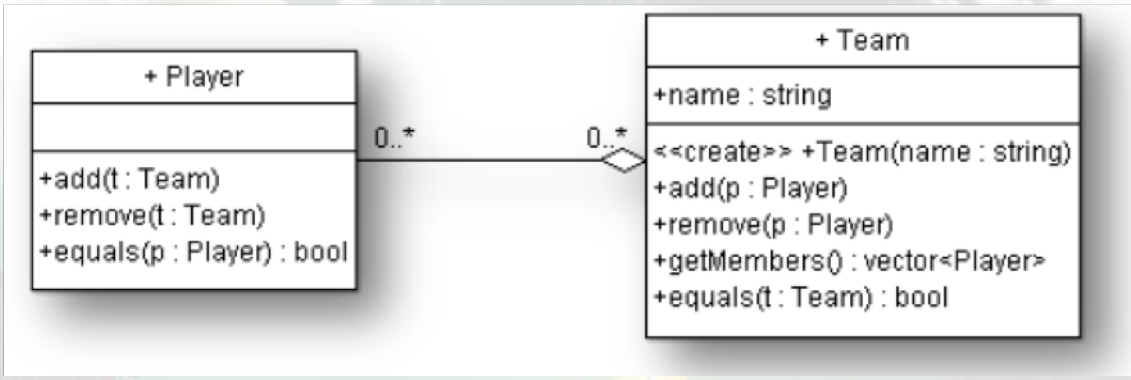


Figura 4: Jugador

És a dir: podrem `afegir`, `esborrar` i `llistar` els jugadors que pertanyen a un equip.

Ademès implementarem un nou mètode per a la realització de les comparacions : implementar el mètode `equals`.

La classe `Team` també tindrà el mètode `toString()` , que servirà per a retornar en forma de cadena els jugadors que formen l'equip.

Fixeu-vos que, quan un jugador s'afegeix a un equip, l'equip s'afegeix alhora a la llista d'equips del jugador. La relació `Team`-`Player` és *bidireccional* i, per tant, `Player` coneix els seus `Team` i `Team` coneix els seus `Player`.

6.2 Es demana

1. Crea la classe `Team` en el paquet joc, amb l'atribut `name`:

- Crea el constructor, passant-li el nom de l'equip
- Per a implementar la relació entre classes definida al diagrama anterior, afegeix un atribut privat de tipus `ArrayList`, anomenat `players`, i els mètodes que apareixen al diagrama per a afegir membres, llevar-los o bé consultar-los.

- També caldrà incloure el mètode `toString()` que retorne el nom de l'equip i les dades dels seus jugadors entre parèntesi. Per exemple:

```
Equip Els Guais:
    John Smith PA:13 / PD:8 / PV:39 (pertany a 2 equips)
    Geronimo PA:9 / PD:4 / PV:100 (pertany a 1 equip)
```

- Afegir el mètode `equals`. El criteri d'igualtat és el contingut de tot l'objecte.

2. Modifica la classe `Player`:

- Per tal que puguen afegir-se els equips d'un jugador caldrà afegir un atribut privat de tipus `ArrayList`, anomenat `teams`, i els mètodes corresponents (per a afegir un jugador a un grup i per a llevar-lo).
- Tin en compte que si assignes un jugador a un grup, automàticament s'ha d'assignar també el grup al jugador, i viceversa (vés en compte: no provoques ***recursió infinita***). També per a llevar un jugador d'un grup.
- El mètode `toString()` s'haurà de modificar per a retornar entre parèntesi la quantitat d'equips als quals pertany el jugador. Per exemple:

```
John Smith PA:13 / PD:8 / PV:39 (pertany a 2 equips)
```

- Afegir el mètode `equals`. El criteri d'igualtat és el nom del jugador i els punts d'atac i defensa i la vida.

3. En la classe `JocDeRol` del paquet inici adapta la funció `provaFase()` per a comprovar el funcionament de les últimes modificacions fetes. Eixa funció ha de crear alguns jugadors i equips. També ha d'assignar jugadors a equips, desassignar-los, etc. També ha de comprovar la igualtat entre alguns `Team` i `Player`.

6.3 Fi de la Fase 4

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 4. Recorda que el nom de l'etiqueta ha de ser `fase-4`.

7 Fase 5. Més relacions

7.1 Explicació

Per tal d'afegir-hi interès, volem implementar en el nostre joc la possibilitat que els jugadors porten armes o escuts que modifiquen la seua capacitat d'atac i de defensa. Cada jugador pot portar múltiples armes, però cada arma pertany a un únic jugador. A dites armes i escuts les anomenarem `Item`

- Cada arma té un nom, un bonus d'atac i un bonus de defensa. Quan un jugador es dispose a atacar, ho farà amb la suma dels seus punts d'atac habituals més la suma de tots els bonus d'atac de les seues armes.
- De la mateixa manera, quan un jugador es dispose a defendre, ho farà amb la suma dels seus punts de defensa habitual més la suma dels bonus de defensa de totes les seues armes.
- Els bonus d'atac i de defensa poden ser negatius i, per tant, penalitzar al jugador.
- Les característiques individuals de cada tipus de jugador (Human, Alien i Warrior) no es modifiquen, sinó que es deixen igual que estaven.

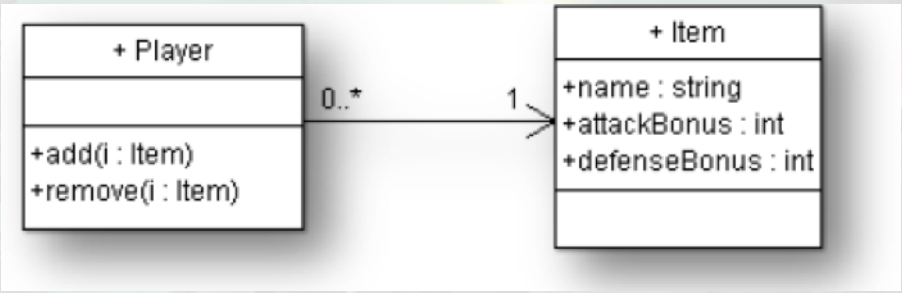


Figura 5: Jugador

7.2 Es demana

1. Crea la classe `Item` en el paquet joc, amb els 3 atributs corresponents i un constructor al qual li passes els 3 paràmetres.
2. Fes els canvis necessaris a la classe `Player` per a implementar el funcionament anterior. És a dir:

- Crea un vector d'ítems anomenat `items`.
- Implementa els mètodes `add` i `remove` per a afegir i llevar un ítem a un jugador.
- A l'hora d'atacar o defendre, augmenta els punts d'atac o de defensa amb la suma dels punts dels items d'un jugador (modificacions en `atack()` i `hit()`)

3. El mètode `toString()` s'haurà de modificar per a retornar també els noms dels seus ítems amb els respectius bonus. Per exemple:

```
John Smith PA:10 / PD:19 / PV:39 (pertany a 1 equip) té els ítems:
  - Sunglasses BA:-1 / BD:-1
  - False Nails BA:5 / BD:2
```

4. En la classe `JocDeRol` del paquet inici adapta la funció `provaFase()` per a comprovar el funcionament.

7.3 Fi de la Fase 5

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 5. Recorda que el nom de l'etiqueta ha de ser `fase-5`.

8 Fase 6. Configuració del programa principal i Excepcions

8.1 Explicació

En el programa principal (en la classe JocDeRol del paquet inici) definirem un ArrayList per a guardar els jugadors, altre per als grups i altre per a les armes. El que volem fer és tota la configuració de donar d'alta els jugadors, items i equips.

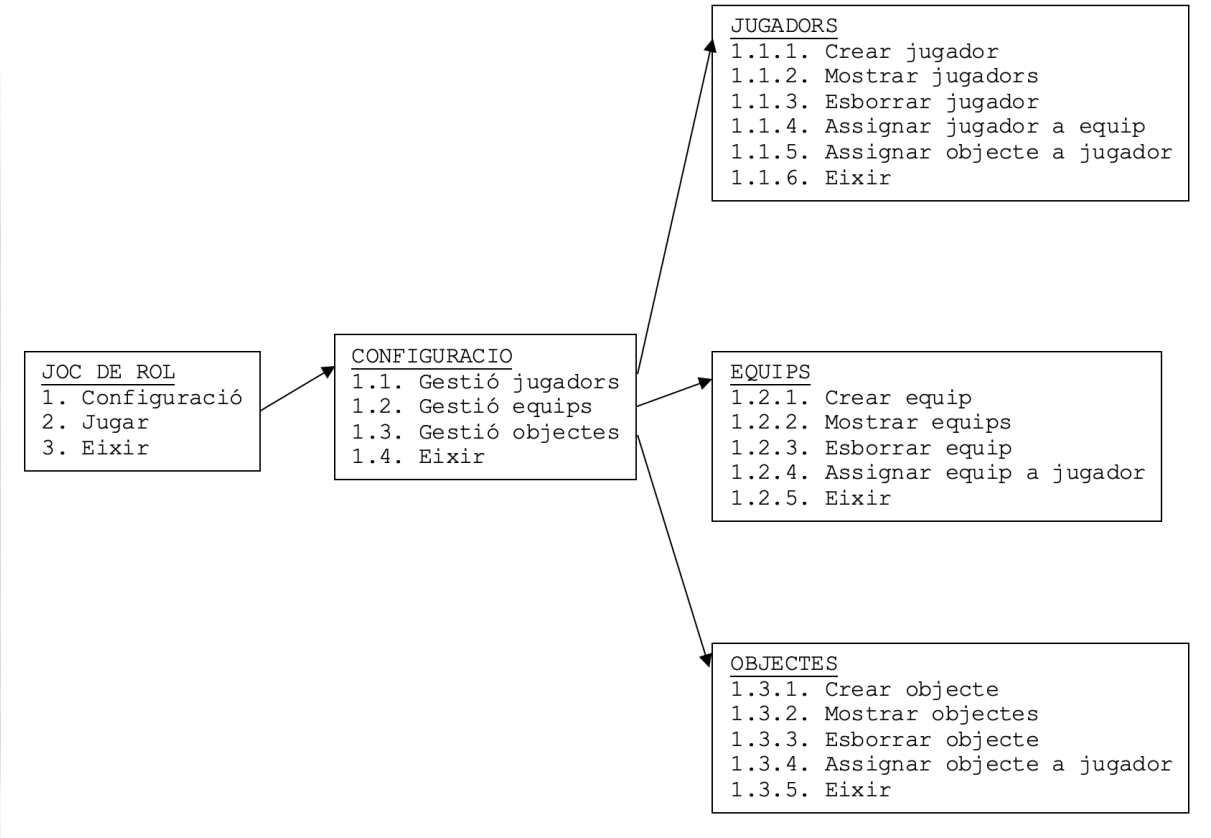


Figura 6: Menú del joc

8.2 Es demana

Fer un programa amb menús que permeta generar tota la estructura del joc.

8.2.1 Menú Jugadors

Dins del menú de jugadors podríem tindre les següents opcions:

```
Crear jugador
Llistar jugadors
Eliminar jugador
...
```

En esta opció crearem tots els objectes de l'aplicació. Per exemple, l'opció de crear jugador podria fer el següent:

1. Preguntar el tipus de jugador (A, W, P), el nom i els punts d'atac (entre 1 i 100)
2. Assignar els punts de defensa (PA + PD = 100) i els punts de vida inicials. Este valor serà igual per a tots els jugadors. Per tant, hauria d'anar com a `static` en la classe corresponent i es podria modificar en l'apartat de configuració. Per defecte, 100.
3. Crear el jugador i posar-lo en l'`ArrayList` de jugadors, comprovant prèviament que no existia ja un jugador *igual*. 2 jugadors seran iguals si tenen el mateix nom. Caldrà implementar el mètode `equals` de la classe `Player`.

Les altres opcions del menú de jugadors (listar, eliminar, etc.) també s'haurien de implementar. La seua funcionalitat és la que el seu nom indica. Per exemple, l'opció de eliminar jugador hauria de preguntar el nom del jugador a eliminar i, si existeix, eliminar-lo de l'`ArrayList` de jugadors.

8.2.2 Menú Equips

Dins del menú d'equips podríem tindre les següents opcions:

```
Crear equip
Llistar equips
Eliminar equip
...
```

Les opcions a implementar són les que el seu nom indica. Per exemple:

- l'opció de crear equip hauria de preguntar el nom de l'equip a crear i, si no existia ja un equip amb eixe nom, crear-lo i posar-lo en l'`ArrayList` d'equips.
- L'opció d'assignar jugador a equip hauria de preguntar el nom del jugador i el nom de l'equip i, si existeixen tots dos, assignar el jugador a l'equip i viceversa.

8.2.3 Menú Ítems

Dins del menú d'ítems podríem tindre les següents opcions:

```
Crear ítem
Llistar ítems
```

```
Eliminar ítem
...
```

Les opcions a implementar són les que el seu nom indica, de manera similar al menú d'equips. Per exemple:

8.2.4 Jugar

Bucle on tots els jugadors van a pegar-se fins quedar sols un en vida:

1. Es comprova que que hi ha almenys 2 jugadors vius. Si no és així, el joc acaba.
2. Es tria de manera aleatòria quin jugador ataca i quin jugador és atacat.
3. Pots fer servir un ArrayList de jugadors vius per a triar de manera aleatòria el jugador que ataca i el jugador atacat.

8.3 Excepcions

Crear i llançar (i capturar) les següents situacions anòmales al llarg del joc. Hauràs de crea les següents excepcions al projecte

1. Un jugador mort no pot atacar ni ser atacat
2. Un jugador no pot atacar-se a ell mateix.
3. No podem llevar d'un equip a un jugador que no li pertany.
4. Un equip no pot tindre jugadors repetits i viceversa.

8.4 Fi de la Fase 6

Fes els commits i etiquetatges que s'han explicat abans per a entregar la fase 6. Recorda que el nom de l'etiqueta ha de ser `fase-6`.

9 Hasta el infitino y más allá

Figura 7: El cel és el límit

De manera addicional, pots tira-li imaginació i crea les teues normes!

- Pots crear més tipus de jugadors, més tipus d'ítems, etc.
- També pots crear un sistema de nivells per a cada jugador, o un sistema de punts d'experiència, o un sistema de classes (màgia, lladres, etc.).
- També pots crear un sistema de combats per equips, o un sistema de combats amb més d'un jugador atacant i més d'un jugador atacat.
- També pots crear un sistema de combats amb més d'una ronda, on els jugadors ataquen i defenen diverses vegades fins que un dels dos jugadors mor.
- També pots crear un sistema de manà, on alguns ítems consumeixen manà
- També pots crear un sistema de sort, on els jugadors tenen una probabilitat de fallar l'atac o la defensa.